

LAB REPORT 8

MECHATRONICS SYSTEM INTEGRATION

DR. ZULKIFLI BIN ZAINAL ABIDIN

SECTION 1

GROUP 6

MATRIC NUMBER	NAME
2312369	MUHAMMAD FAHIM BIN AHMAD NORISHAM
2312479	MUHAMMAD HAZIMUDDIN BIN FAIRUZ
2312451	MUHAMMAD ZAMARUL AZIM BIN ZULKHIBRI

DATE OF SUBMISSION

Wednesday, 17th December 2025

Abstract

This project presents the design and implementation of a color detection system using an Arduino Uno and the Gravity HuskyLens AI Vision Camera. The system was developed to recognize and classify different colors that were previously trained and stored in the HuskyLens internal database. Once a trained color is detected, the HuskyLens transmits the corresponding color ID to the Arduino through a serial communication interface. The Arduino then processes this data and activates an RGB LED, displaying a color that matches the detected object in real time.

The primary objective of this study was to evaluate the accuracy, consistency, and response time of the HuskyLens when performing color detection under varying environmental conditions. Several test scenarios were conducted, including changes in ambient lighting and object distance, to observe how these factors influenced system performance. The results showed that the HuskyLens maintained reliable and stable color recognition when properly calibrated and operated under controlled lighting conditions. Although slight variations in brightness and distance caused small fluctuations in detection accuracy, the system continued to operate effectively in most practical situations.

This project demonstrates the practical application of AI vision sensors in embedded systems and highlights their potential use in real-time automation, robotics, and smart sensing applications.

Keywords: Arduino, HuskyLens, Color Detection, RGB LED, AI Vision, Embedded Systems

Table of Contents

Abstract.....	1
1. Introduction.....	3
2. Materials and Equipments.....	4
3. Experimental Setup.....	5
4. Methodology.....	6
5. Data Collection.....	10
6. Data Analysis.....	11
7. Results.....	12
8. Discussion.....	13
9. Conclusion.....	14
10. Recommendations.....	15
11. References.....	16
Appendix A - Circuit Diagram.....	17
Appendix B - Coding Snippets.....	18
Appendix C - Results.....	21
Acknowledgments.....	23

WEEK 9: IMAGE/VIDEO INPUT INTERFACING WITH MICROCONTROLLER

1. Introduction

Color detection is important in many automation systems such as object sorting, traffic monitoring, and industrial inspection. Traditional color sensors often rely on simple RGB light measurements and are highly affected by changes in lighting, distance, and surface texture, which can reduce accuracy.

To address these limitations, this project uses the Gravity HuskyLens AI Vision Camera, a compact vision sensor with built-in artificial intelligence. Unlike basic color sensors, HuskyLens processes images internally and can be easily trained using its onboard buttons and display to recognize colors, objects, and patterns without external software.

The HuskyLens supports communication interfaces such as I2C and UART, making it compatible with microcontrollers like the Arduino Uno. In this project, UART communication was used to transmit detected color ID data from the HuskyLens to the Arduino, which then activates an RGB LED to display the corresponding color in real time. HuskyLens board overview shown in Figure 1.0 below.

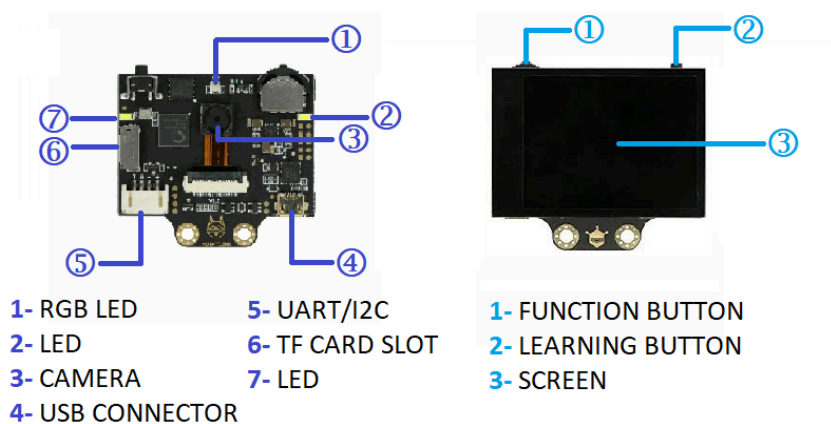


Figure 1.0 Shows overview of the HuskyLens

Our main objectives in this experiment were:

- To design and build a color detection system using Arduino Uno and HuskyLens
- To train the HuskyLens to recognize five predefined colors
- To interface the HuskyLens with Arduino using serial communication
- To display detected colors using an RGB LED
- To evaluate detection accuracy under different lighting and distance conditions
- To demonstrate practical applications of AI vision sensors in embedded systems

2. Materials and Equipments

Gravity HuskyLens AI Vision Camera

1

RGB LED Module - HW479

1

Breadboard

Jumper Wires



Figure 2.0 Shows the RGB LED - HW479

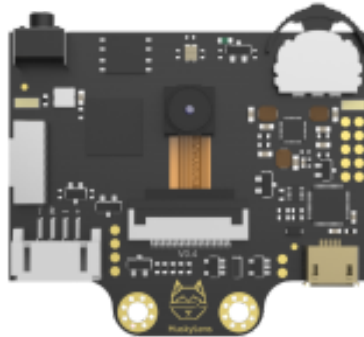


Figure 1.0 Shows the HuskyLens AI Vision Camera

3. Experimental Setup

The experimental setup focuses on correctly connecting the HuskyLens AI Vision Camera with the Arduino Uno and preparing the system for stable color detection. The main purpose of this setup is to make sure that data can be transferred reliably from the camera to the microcontroller and that the RGB LED can respond correctly based on the detected colors.

First, the HuskyLens camera was placed on a flat surface and connected to the Arduino Uno using jumper wires. The UART communication method was selected because it is simple and reliable for beginners. Since the Arduino Uno only has one hardware serial port, the SoftwareSerial library was used to create an additional serial communication channel. The TX and RX pins of the camera were connected to digital pins 4 and 5 of the Arduino.

The RGB LED was placed on the breadboard and connected to three digital output pins of the Arduino through resistors. Each resistor helps protect the LED from drawing too much current. The common pin of the RGB LED was connected to ground.

During the setup process, special attention was paid to wire placement to avoid loose connections, which could cause unstable readings or system errors.

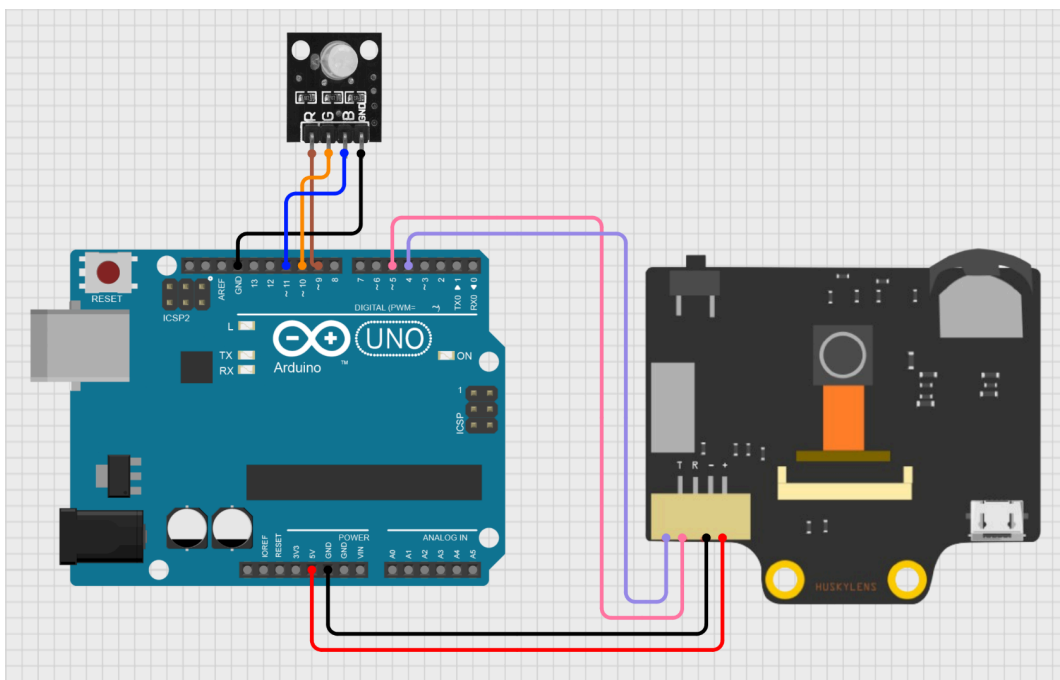


Figure 3.0 Shows the schematic setup using Cirkit Software

4. Methodology

This section describes the step-by-step procedure used to build and test the color detection system. The main objective was to train the HuskyLens to recognize specific colors and to program the Arduino Uno to respond accurately to the detected results.

4.1. Hardware Setup

The hardware components used in this project were the Arduino Uno, Gravity HuskyLens AI Vision Camera, and an HW-479 RGB LED module. All devices were powered using the Arduino Uno's 5V and GND pins. The HuskyLens was connected to the Arduino using UART communication, where the TX pin of the HuskyLens was connected to the RX pin of the Arduino, and the RX pin of the HuskyLens was connected to the TX pin of the Arduino. The HW-479 RGB LED module was connected to three digital output pins of the Arduino for controlling the red, green, and blue channels.

The HuskyLens camera was mounted securely and positioned to face the test objects at a fixed distance to ensure consistent detection conditions. The hardware setup focused on connecting the Arduino UNO with the HuskyLens and the HW-79 RGB LED. The following connections were made:

Husky Lens	VCC → 5V
	GND → GND
	TX → D4 (SoftwareSerial RX)
	RX → D5 (SoftwareSerial TX)
HW-479 RGB LED	GND → GND
	R → D9
	G → D10
	B → D11

Table 4.1 Shows the connection for the HuskyLens and HW-79 RGB LED pin with the Arduino Uno pin.

4.2. Programming the Arduino through ArduinoIDE

```
#include "HUSKYLENS.h"
#include "SoftwareSerial.h"

HUSKYLENS huskylens;
SoftwareSerial mySerial(4, 5); // RX, TX

// RGB LED pins (HW-479)
#define RED_PIN 9
#define GREEN_PIN 10
#define BLUE_PIN 11

void printResult(HUSKYLENSResult result);
void setRGB(int r, int g, int b);

void setup() {
    Serial.begin(115200);
    mySerial.begin(9600);

    // RGB pins
    pinMode(RED_PIN, OUTPUT);
    pinMode(GREEN_PIN, OUTPUT);
    pinMode(BLUE_PIN, OUTPUT);

    // Turn off LED at startup
    setRGB(0, 0, 0);

    while (!huskylens.begin(mySerial)) {
        Serial.println(F("Begin failed!"));
        Serial.println(F("1. Check Protocol Type = Serial
9600"));
        Serial.println(F("2. Check wiring"));
        delay(200);
    }

    Serial.println("HuskyLens Ready - Multi Color + RGB LED
Enabled!");
}
```

```

void loop() {
    if (!huskylens.request()) {
        Serial.println("Fail to request data from
HuskyLens!");
    }
    else if (!huskylens.isLearned()) {
        Serial.println("Nothing learned! Please train
colors.");
    }
    else if (!huskylens.available()) {
        Serial.println("No object detected.");
        setRGB(0, 0, 0); // LED OFF when nothing detected
    }
    else {
        while (huskylens.available()) {
            HUSKYLENSResult result = huskylens.read();
            printResult(result);
        }
    }
}

void printResult(HUSKYLENSResult result){
    if (result.command == COMMAND_RETURN_BLOCK){

        Serial.print("Detected ID = ");
        Serial.println(result.ID);

        // RGB LED reacts based on color ID
        switch(result.ID) {

            case 1: // RED
                Serial.println("Color: RED");
                setRGB(255, 0, 0);
                break;

            case 2: // BLUE
                Serial.println("Color: BLUE");
                setRGB(0, 0, 255);
                break;

```

```

        case 3: // GREEN
            Serial.println("Color: GREEN");
            setRGB(0, 255, 0);
            break;

        case 4: // YELLOW
            Serial.println("Color: YELLOW");
            setRGB(255, 255, 0);
            break;

        default:
            Serial.println("Unknown ID - LED OFF");
            setRGB(0, 0, 0);
            break;
    }
}

// RGB LED HELPER (common cathode)
// Send values 0-255 for brightness
void setRGB(int r, int g, int b) {
    analogWrite(RED_PIN, r);
    analogWrite(GREEN_PIN, g);
    analogWrite(BLUE_PIN, b);
}

```

4.3. System Testing

System testing was conducted after both the hardware and software were completed. Different colored objects were presented in front of the HuskyLens to verify the system's accuracy. The responses of the RGB LED and the data shown in the Serial Monitor were carefully observed and recorded.

Multiple trials were performed under different lighting conditions and object distances to evaluate the reliability of the system. The system was considered successful when the RGB LED consistently matched the detected color and when the Serial Monitor displayed correct and stable data from the HuskyLens.

5. Data Collection

Data collection was carried out by testing the system using different colored objects that had been trained earlier in the HuskyLens Color Recognition mode. Five colors were learned and stored in the HuskyLens database, namely red (ID 1), blue (ID 2), green (ID 3), yellow (ID 4), and purple (ID 5). During testing, each colored object was placed in front of the HuskyLens at a fixed distance, and the detected color ID was observed through the Serial Monitor.

At the same time, the response of the HW-479 RGB LED was monitored to verify whether it displayed the correct color corresponding to the detected ID. Several trials were conducted to observe system behavior under different ambient lighting conditions, such as normal indoor lighting and slightly dimmer or brighter environments. The system response time and detection consistency were also observed visually.

Overall, the collected data focused on detection accuracy, response time of the RGB LED, and the effect of lighting conditions on color recognition performance.

Trial	Actual Color	Detected ID	RGB Display	Correct Detection
1	Red	1	Red	Yes
2	Blue	2	Blue	Yes
3	Green	3	Green	Yes
4	Yellow	4	Mix of red and green	Yes
5	Purple	5	Mix of red and blue (Purple)	Yes

Table 5.0 Shows the Sample Data Collected During Color Detection Testing

6. Data Analysis

The collected observations indicate that the **HuskyLens** was able to detect the trained colors accurately when the lighting conditions were stable and sufficient. The detected color IDs were compared with the actual colors presented to the HuskyLens in order to evaluate the accuracy and reliability of the system. The response of the HW-479 RGB LED was also analyzed to determine whether the Arduino processed the detected data correctly.

Based on Table **5.0**, the analysis shows that the HuskyLens was able to recognize most of the trained colors accurately when the lighting conditions were stable. For each correct detection, the Arduino received the correct color ID and activated the corresponding RGB LED color without noticeable delay. This indicates that the serial communication between the HuskyLens and Arduino Uno was working properly and consistently.

However, the collected data also shows that changes in ambient lighting affected system performance. In brighter or dimmer environments, the HuskyLens occasionally required the colored object to be repositioned or held steady for a longer period before a correct detection was made. These situations sometimes resulted in a slight delay in LED response or temporary failure to detect the object.

Despite these variations, the overall consistency of the system remained high. When objects were placed at a suitable distance and lighting conditions were controlled, the system produced stable and repeatable results across multiple trials. This analysis confirms that while the AI-based vision system is reliable, proper environmental conditions play an important role in achieving optimal performance.

7. Results

The results of this experiment demonstrate as shown in **Table 5.0** that the color detection system using the HuskyLens AI Vision Camera and Arduino Uno was successfully implemented. Based on the collected and analyzed data, the system was able to accurately detect the trained colors and display the correct output using the RGB LED.

Throughout the testing process, the HuskyLens consistently identified the five trained colors red, blue, green, yellow, and purple. It is also transmitted the correct color IDs to the Arduino via serial communication. The Arduino then processed this information and activated the appropriate RGB LED channel. In most cases, the LED response was immediate, indicating efficient data processing and system responsiveness.

The system performed best under normal indoor lighting conditions, where detection accuracy and LED response were stable. Minor detection issues were observed when lighting conditions changed significantly, such as in overly bright or dim environments. However, these issues did not prevent the system from functioning and were resolved by adjusting the object position or lighting conditions.

Overall, the experiment met all its intended objectives and confirmed that the integration of the HuskyLens, Arduino Uno, and RGB LED module can provide an effective real-time color detection system using AI vision technology.

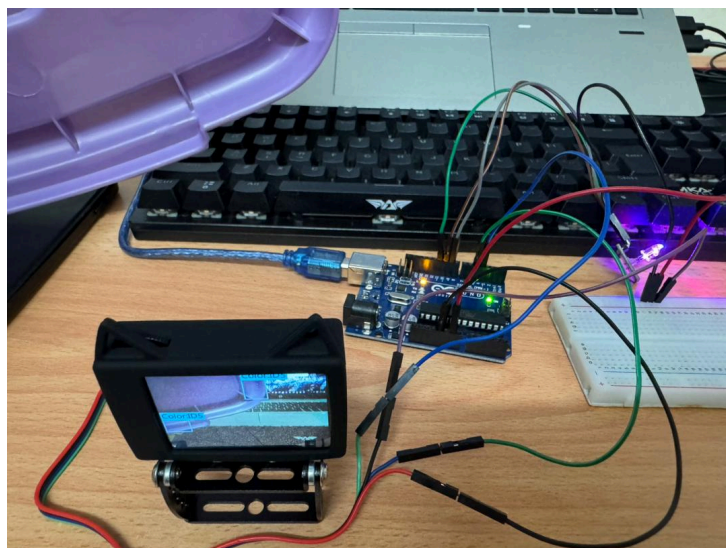


Figure 7.0 Shows the HW-479 turn purple based on the color detected by the HuskyLens AI

8. Discussion

This experiment demonstrated the effectiveness of integrating an AI vision sensor with a microcontroller for real-time color detection. The HuskyLens AI Vision Camera showed accurate and strong performance in recognizing trained colors and providing reliable output data to HW-479 through the Arduino Uno. The immediate response of the RGB LED indicates efficient serial communication and fast processing, which is essential for real-time embedded systems.

One key finding from the experiment is that proper training and calibration greatly improved detection accuracy. When the colors were trained under the same lighting conditions used during testing, the HuskyLens was able to detect colors more consistently. The system performed best when the object was placed at an appropriate distance and held steady in front of the camera.

Additionally, there is one important observation and challenge from the experiment is the influence of ambient lighting on detection accuracy. While the HuskyLens performed well under consistent lighting conditions, variations in brightness affected its ability to detect colors reliably. This limitation behaviour is common in vision-based systems, as changes in illumination can alter how colors appear to the camera. Despite this limitation, the system remained responsive and functional, especially when the lighting conditions were properly adjusted.

Despite these challenges, the HuskyLens still showed strong performance compared to traditional color detection methods, as it allows multiple colors to be trained easily and provides stable detection results without complex programming. For future improvements, one of the ways to improve is by set-up the system or placing it in a controlled lighting environment to minimize the detection errors.

Overall, the experiment highlights the advantages of AI-based vision systems compared to traditional color sensors. The ability to train multiple colors easily and obtain stable detection results makes the HuskyLens suitable for applications such as automation, robotics, and smart sensing systems.

9. Conclusion

In conclusion, this experiment successfully constructed, designed and implemented a color detection system using the Arduino Uno and the Gravity HuskyLens AI Vision Camera. The system was able to accurately recognize multiple trained colors and respond by displaying the detected color through an RGB LED in real time. The fast LED response confirmed that the communication between the HuskyLens and Arduino was reliable and suitable for real-time embedded applications.

The experiment also highlighted the importance of proper training and lighting conditions in vision-based systems. The HuskyLens performed consistently under stable indoor lighting, while changes in ambient brightness affected detection accuracy. Despite these limitations, the system met all the intended objectives and demonstrated reliable performance when operated under controlled conditions.

Overall, this project provided valuable hands-on experience in integrating AI vision sensors with microcontrollers, serial communication, and basic automation concepts. The results show that the HuskyLens is an effective and user-friendly solution for color detection applications and can be applied to future projects in robotics, automation, and smart sensing systems.

10. Recommendations

Based on the experiment, several improvements can be suggested for future work:

- **Lighting Control:**

The system should be operated in a controlled or enclosed lighting environment to minimize the effect of changing ambient light. Using consistent light sources such as LED lighting can help reduce shadows and reflections, which will improve color detection accuracy.

- **Re-training Under Different Conditions:**

The HuskyLens can be trained using the same colors under different lighting conditions, such as bright and dim environments. This would allow the camera to better recognize colors in various situations and improve system reliability.

- **Hardware Expansion:**

Additional components such as servo motors, DC motors, or relays can be integrated into the system. This would allow the project to demonstrate real-world applications, such as automatic color-based object sorting or robotic movement.

- **Advanced Processing:**

Future versions of the system could explore more advanced AI or image processing methods to enhance detection speed and accuracy. Combining the HuskyLens with other sensors could also improve decision-making in more complex applications.

11. References

Dejan. (2018, August 4). *Arduino Color Sensing Tutorial - TCS230 TCS3200 Color Sensor - HowToMechatronics*.HowToMechatronics.

<https://howtomechatronics.com/tutorials/arduino/arduino-color-sensing-tutorial-tcs230-tcs3200-color-sensor/>

DFRobot. (n.d.-a). *Gravity: HUSKYLENS - An Easy-to-use AI Machine Vision Sensor*. Wiki.dfrobot.com.

https://wiki.dfrobot.com/HUSKYLENS_V1.0_SKU_SEN0305_SEN0336

DFRobot. (n.d.-b). *HuskyLens – An AI Camera: Click, Learn, and Play | Vision Sensor | DFRobotElectronics*. Www.dfrobot.com. <https://www.dfrobot.com/product-1922.html>

DFRobot. (2019, July 24). *HuskyLens – An AI Camera: Click, Learn, and Play! Projects and Tutorials - DFRobot*; DFRobot. <https://www.dfrobot.com/blog-1327.html>

HowToMechatronics. (2016a). *Arduino Color Sensing Tutorial - TCS230 TCS3200 Color Sensor*. Www.youtube.com. <https://www.youtube.com/watch?v=CPUXxuyd9xw>

HowToMechatronics. (2016b). *Arduino Color Sorter Project*. Www.youtube.com. <https://www.youtube.com/watch?v=g3i51hdfLaw>

RootSaid. (2020, March 20). *HuskyLens Tutorial - Getting Started with Husky Lens - AI Vision Sensor*. Www.youtube.com. <https://www.youtube.com/watch?v=HZxFj2u9k00>

Appendix A - Circuit Diagram

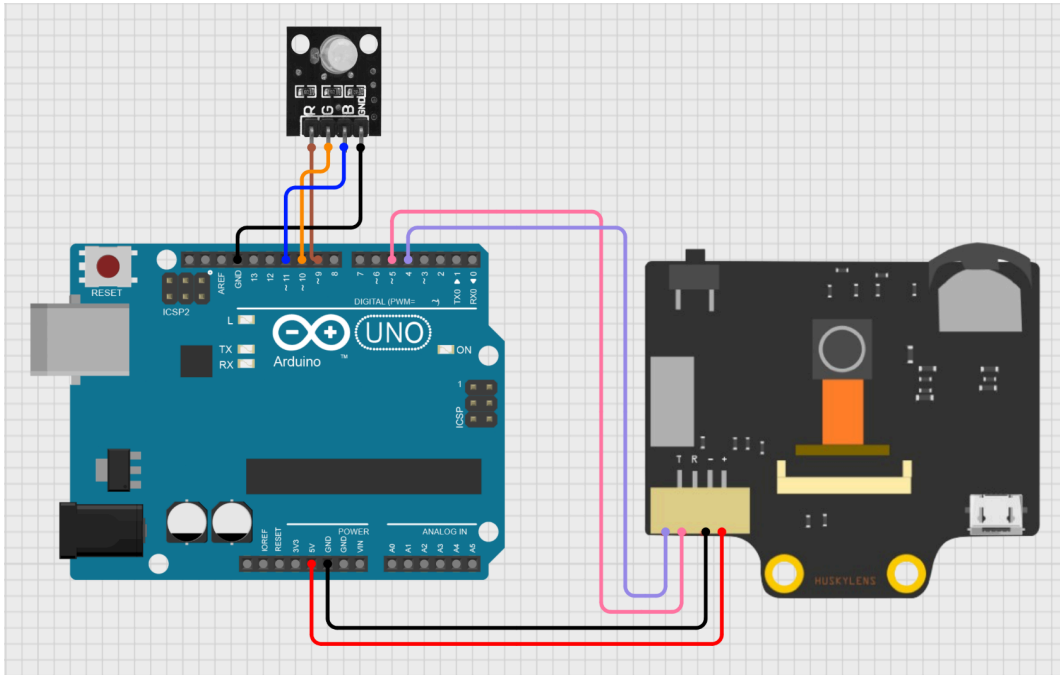


Figure A1 Shows the connection of circuit configuration by using Cirkit Software

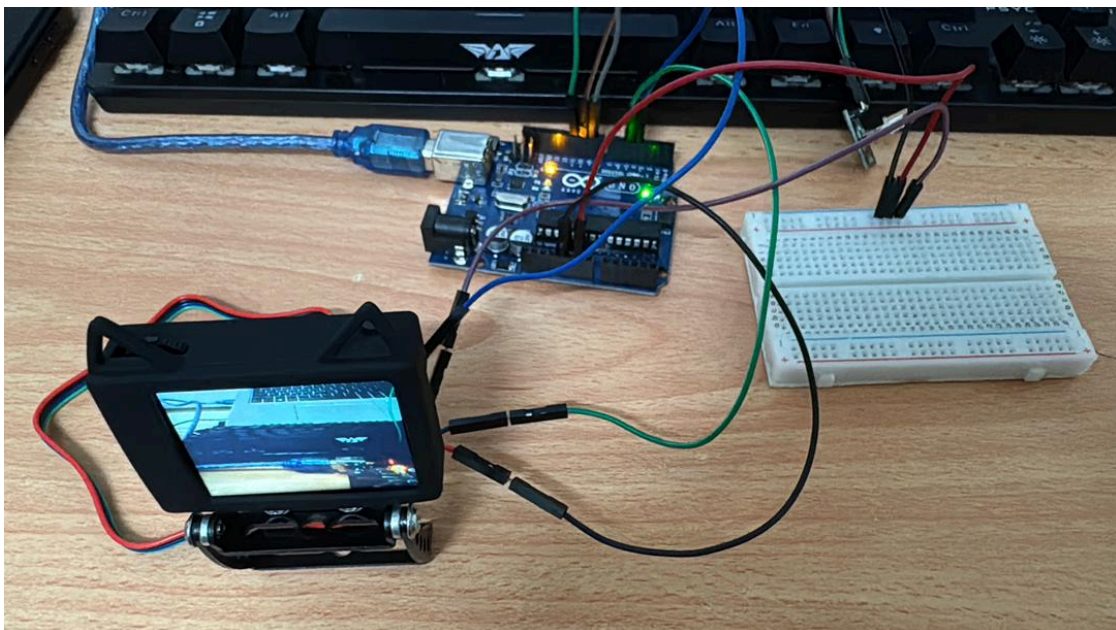


Figure A2 Shows the real connection of circuit hardware

Appendix B - Coding Snippets

```
#include "HUSKYLENS.h"
#include "SoftwareSerial.h"

HUSKYLENS huskylens;
SoftwareSerial mySerial(4, 5); // RX, TX

// RGB LED pins (HW-479)
#define RED_PIN 9
#define GREEN_PIN 10
#define BLUE_PIN 11

void printResult(HUSKYLENSResult result);
void setRGB(int r, int g, int b);

void setup() {
    Serial.begin(115200);
    mySerial.begin(9600);

    // RGB pins
    pinMode(RED_PIN, OUTPUT);
    pinMode(GREEN_PIN, OUTPUT);
    pinMode(BLUE_PIN, OUTPUT);

    // Turn off LED at startup
    setRGB(0, 0, 0);

    while (!huskylens.begin(mySerial)) {
        Serial.println(F("Begin failed!"));
        Serial.println(F("1. Check Protocol Type = Serial
9600"));
        Serial.println(F("2. Check wiring"));
        delay(200);
    }

    Serial.println("HuskyLens Ready - Multi Color + RGB LED
Enabled!");
}
```

```

void loop() {
    if (!huskylens.request()) {
        Serial.println("Fail to request data from
HuskyLens!");
    }
    else if (!huskylens.isLearned()) {
        Serial.println("Nothing learned! Please train
colors.");
    }
    else if (!huskylens.available()) {
        Serial.println("No object detected.");
        setRGB(0, 0, 0); // LED OFF when nothing detected
    }
    else {
        while (huskylens.available()) {
            HUSKYLENSResult result = huskylens.read();
            printResult(result);
        }
    }
}

void printResult(HUSKYLENSResult result){
    if (result.command == COMMAND_RETURN_BLOCK){

        Serial.print("Detected ID = ");
        Serial.println(result.ID);

        // RGB LED reacts based on color ID
        switch(result.ID) {

            case 1: // RED
                Serial.println("Color: RED");
                setRGB(255, 0, 0);
                break;

            case 2: // BLUE
                Serial.println("Color: BLUE");
                setRGB(0, 0, 255);
                break;

```



```
        case 3: // GREEN
            Serial.println("Color: GREEN");
            setRGB(0, 255, 0);
            break;

        case 4: // YELLOW
            Serial.println("Color: YELLOW");
            setRGB(255, 255, 0);
            break;

        default:
            Serial.println("Unknown ID - LED OFF");
            setRGB(0, 0, 0);
            break;
    }
}

// RGB LED HELPER (common cathode)
// Send values 0-255 for brightness

void setRGB(int r, int g, int b) {
    analogWrite(RED_PIN, r);
    analogWrite(GREEN_PIN, g);
    analogWrite(BLUE_PIN, b);
}
```

Appendix C - Results

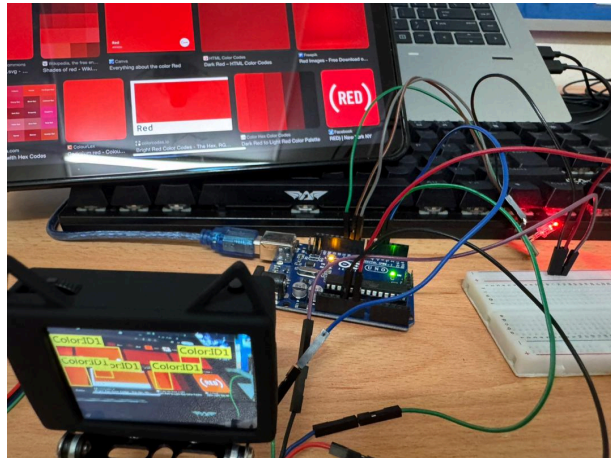


Figure C1 Shows the HW-479 turn on RED

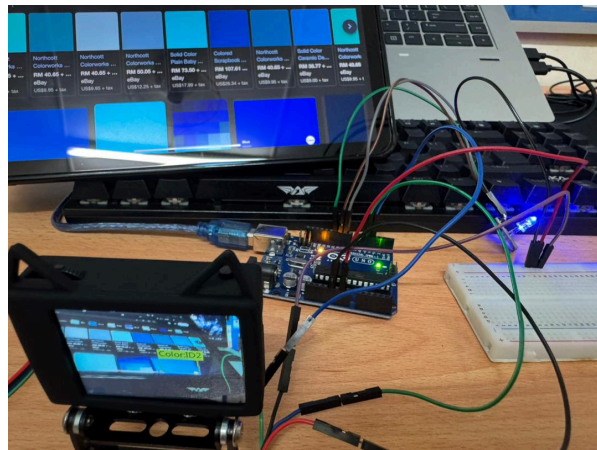


Figure C2 Shows the HW-479 turn on BLUE

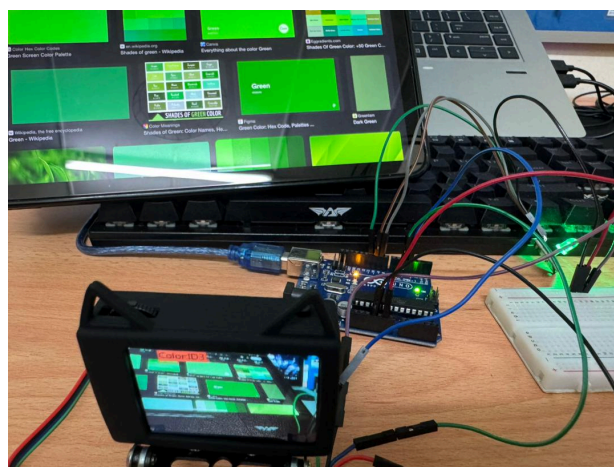


Figure C2 Shows the HW-479 turn on GREEN

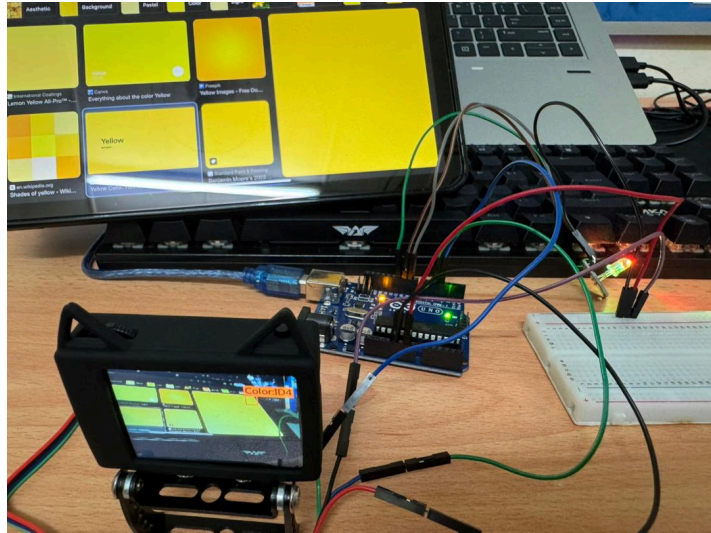


Figure C2 Shows the HW-479 turn on YELLOW (Mix of RED and GREEN)

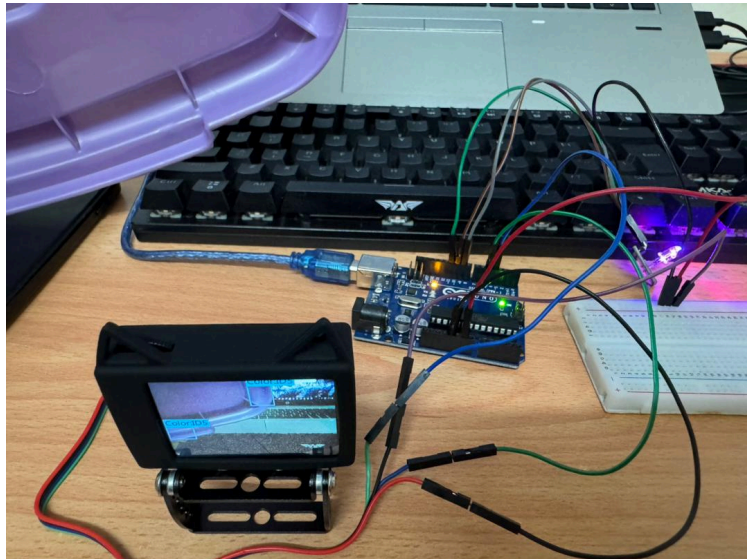


Figure C2 Shows the HW-479 turn on PURPLE (Mix of RED and BLUE)

Acknowledgments

We would like to express our sincere gratitude to all those who supported us during this laboratory experiment. First and foremost, we would like to thank our lecturer for providing clear guidance, instructions, and valuable advice throughout the experiment. Their explanations helped us understand how to integrate the HuskyLens AI Vision Camera with the Arduino, calibrate colors, and control the RGB LED effectively.

We would also like to thank our classmates for their support and encouragement. Discussing ideas, sharing troubleshooting tips, and assisting with hardware connections made the experiment easier and more enjoyable. Their help was especially valuable when setting up the breadboard, wiring the RGB LED, and testing the color detection system.

We are grateful for the availability of the laboratory equipment, including the HuskyLens, Arduino Uno, RGB LED, breadboard, and jumper wires. Without these materials, completing the experiment successfully would not have been possible.

Finally, we would like to acknowledge our family and friends for their encouragement and patience while we prepared this report. Their support motivated us to complete the experiment thoroughly and submit a detailed lab report.

This experiment was a valuable learning experience, allowing us to develop practical skills in programming, AI vision sensor integration, and real-time color detection, which will be useful in future electronics and robotics projects.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specific in references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or a person.

We hereby certify that this report has **not been done by only one individual and all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We, therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature: 

Name: Muhammad Fahim Bin Ahmad Norisham

Matric Number: 2312369

Read /

Understand /

Agree /

Signature: 

Name: Muhammad Zamarul Azim Bin Zulkhibri

Matric Number: 2312451

Read /

Understand /

Agree /

Signature: 

Name: Muhammad Hazimuddin Bin Fairuz

Matric Number: 2312479

Read /

Understand /

Agree /